

Deep Generative Image Modeling

Term-Project Title: Application of AE and GAN in MONAI

Group 8 : 徐綉惠、王家駿、林俞琛

I. Motivation

In medical cases, image data is often limited due to the rarity of certain diseases. Reconstructing medical images by removing blur and noise using AutoEncoders (AEs) or generating high-resolution and diverse medical images using Generative Adversarial Networks (GANs) can produce rich datasets that can be used to train deep learning models for the medical domain.

II. Description

In this project, we used MONAI to implement AE and GAN respectively on the MedNIST dataset.

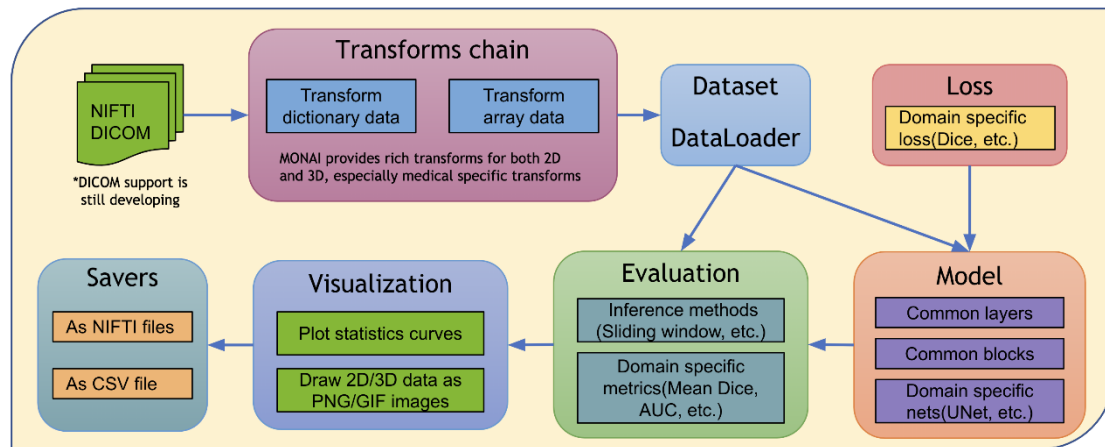
(1) MONAI (Medical Open Network for AI)

MONAI is an open-source framework based on PyTorch, specifically designed for developing deep learning models in medical imaging. It provides a comprehensive toolkit for various tasks, including segmentation, classification, and registration. The framework features domain-specific tools such as 3D data handling, spatial transformations, and medical data loaders. Additionally, it includes a Model Zoo that offers pre-trained models for tasks like organ segmentation and tumor detection. MONAI supports extensive data augmentation techniques, including affine transformations, intensity adjustments, and elastic deformations. It is optimized for GPU acceleration, allowing it to efficiently handle large 3D datasets. Common applications of MONAI include medical image segmentation, disease classification, and image registration across different modalities. Widely utilized in both research and clinical settings, MONAI accelerates AI-driven healthcare innovations by facilitating robust and scalable model development.

(2) MedNIST Dataset

The data was collected from multiple sources, including TCIA, the RSNA Bone Age Challenge, and the NIH Chest X-ray dataset.

The typical example of the end-to-end workflow in the field of medical deep learning.

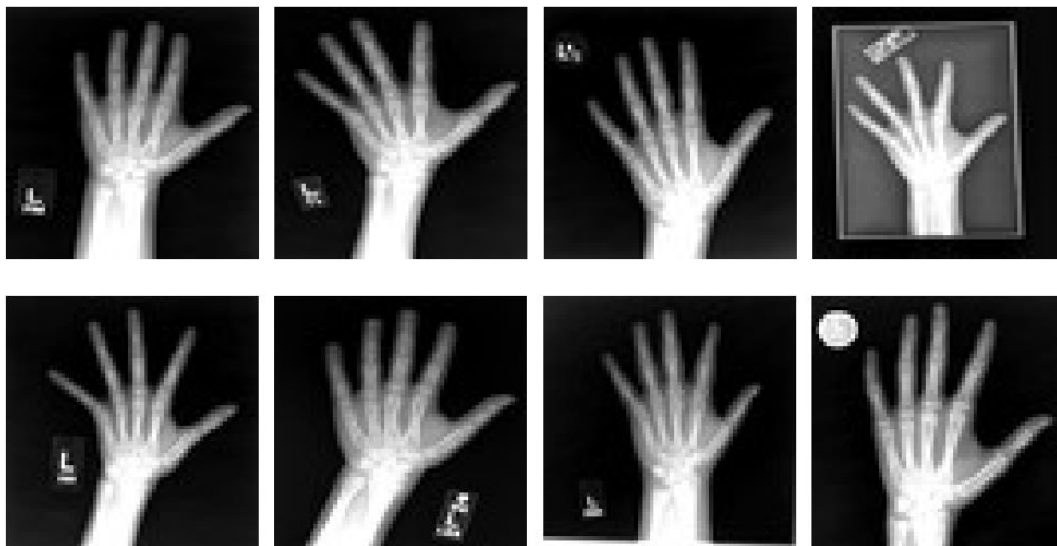


III. Implementation (Our Plan Approaches)

(1) Data Preprocessing

Step 1. Specify the **scan type** as **"Hand"** to extract all hand images from the dataset **(for both AE and GAN).**

Step 2. Shuffle all the collected "hand" images randomly **(only for AE).**



Step 3. Dataset is divided into **80%** for **training** and **20%** for **testing (only for AE).**

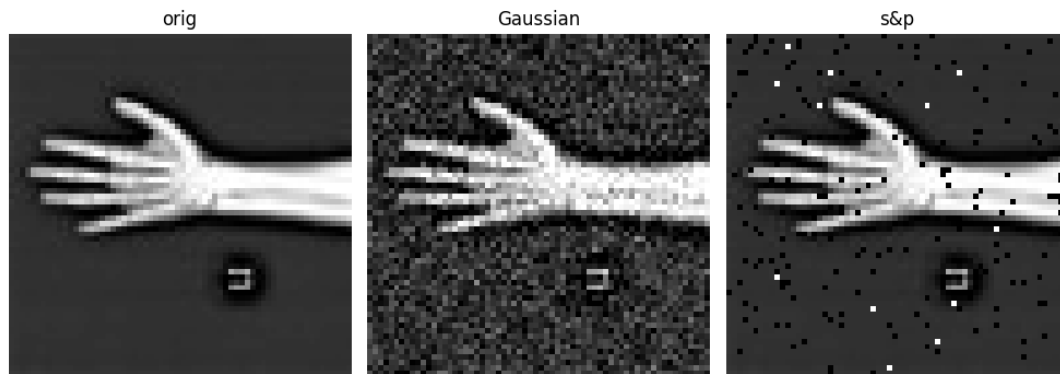
Total number of images: 10,000.

Number of images for training is 8,000.

Number of images for testing is 2,000.

Step 4. Add two types of noise to the original image (**only for AE**).

In order to train the AE to perform deblurring and denoising of images, we need to create degraded images as inputs. Therefore, we will add **Gaussian noise** and **Salt-and-Pepper noise** to the original image, respectively.



Step 5. Define the transformations used for **image augmentation** (**for both AE and GAN**).

Scale pixel intensities to a range of $[0, 1]$.

Randomly rotate the image within ± 15 degrees.

Flip the image vertically at random.

Randomly zoom in or out between 90% and 110%.

Note: Image augmentation is applied only to the training data and not to the testing data (in the AE case).

Step 6. Build the **DataLoader** for the training and testing datasets (**for both AE and GAN**).

The batches of 300 images are loaded in parallel using 10 worker processes, with shuffling set to True, ensuring that the order of images is randomized in each epoch to enhance training robustness.

(2) Training workflow of the AE

Step 1. Define the checkpoint function.

This function ensures that training can resume from the last saved state if it is interrupted. Additionally, it supports training on various types of noisy data.

Step 2. Initialize the AE model architecture.

Spatial Dimension:

This parameter, set to **2**, indicates that the AE is designed to process 2D images, such as grayscale or RGB images. It ensures that the convolutional and pooling operations are applied across two spatial dimensions: height and width.

In/Out Channels:

These parameters are set to **1**, specifying that the input and output of the AE are single-channel grayscale images. This setup is commonly used for grayscale image reconstruction tasks.

Channels:

This specifies the number of output filters for each convolutional layer in the encoder. The model consists of four convolutional layers in the encoder: the first layer produces **4** feature maps, the second outputs **8**, the third generates **16**, and the fourth yields **32**. Increasing the number of channels at each layer allows the network to learn progressively more complex and abstract features, which are essential for capturing fine-grained details.

Strides:

This specifies the step size for each convolutional layer in the encoder. Each value corresponds to a convolutional layer, with a stride of **2** indicating that the filter moves 2 pixels at a time. This effectively reduces the spatial dimensions (height and width) by half after each layer, allowing the network to compress the input image into a more compact latent space representation.

Step 3. Select the appropriate loss function and optimizer.

Mean Squared Error (MSE) Loss:

The MSE loss quantifies reconstruction accuracy by averaging the squared differences between each pixel in the reconstructed image produced by the AE and the corresponding pixel in the original image.

Formula

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i is the original pixel value, $\hat{y}_i$ is the reconstructed pixel value, and n is the total number of pixels.

Advantages

Pixel-wise Comparison: MSE loss emphasizes pixel-to-pixel accuracy, making it particularly suitable for tasks that require precise reconstruction of the original image.

Penalizing Large Errors: Squaring the error amplifies larger discrepancies, ensuring that the model prioritizes correcting substantial reconstruction errors.

Smooth Loss Surface: MSE loss provides a smooth gradient for optimization, which helps in stable convergence during the training process.

Adaptive Moment Estimation (Adam) Optimizer:

Adam optimizer adjusts model weights during training by combining the benefits of both momentum and adaptive learning rate methods. It maintains two moving averages throughout the training process: the first moment estimate (mean), which tracks the average of the gradients (momentum), and the second moment estimate (variance), which monitors the average of the squared gradients (adaptive learning rate).

Formula

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

where θ_t are the model parameters, $\hat{m}_t$ is the corrected mean of gradients, $\hat{v}_t$ is the corrected variance, η is the learning rate, and ϵ is a small constant to avoid division by zero.

In this project, the **learning rate** of the AE is set to **0.001**.

Advantages

Adaptive Learning Rates: Adam adjusts the learning rate for each parameter individually, making it particularly effective for model such as AE, which often contain numerous layers and parameters.

Efficient Convergence: The Adam optimizer converges more quickly than standard stochastic gradient descent (SGD) by integrating momentum with adaptive learning rates.

Robust to Noisy Gradients: It handles noisy gradient estimates well, which is useful when working with small batch sizes.

Step 4. Training for different data types.

Variants: The AE is trained separately for three input types:

"orig": Original images.

"gaus": Images with Gaussian noise.

"s&p": Images with Salt-and-Pepper noise.

Overall

The AE is trained for **50 epochs** on each dataset variant, saving checkpoints every 10 epochs and tracking the average epoch loss for analysis.

Data Type: "orig"

Epochs	Average Loss
10	0.0231
20	0.0176
30	0.0152
40	0.0130
50	0.0097

Data Type: "gaus"

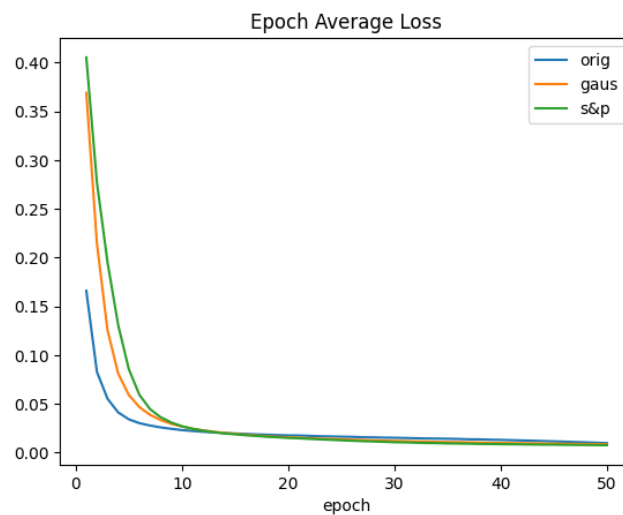
Epochs	Average Loss
10	0.0268
20	0.0156
30	0.0119
40	0.0098
50	0.0085

Data Type: "s&p"

Epochs	Average Loss
10	0.0270
20	0.0150
30	0.0106
40	0.0086
50	0.0076

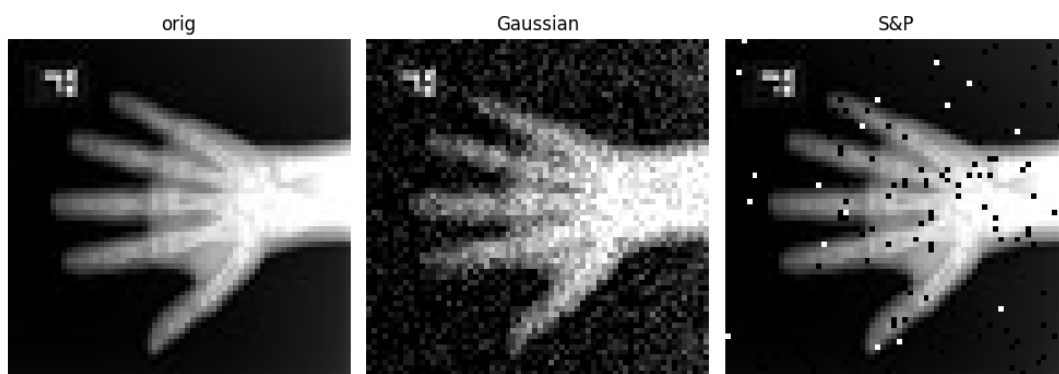
Step 5. Plot loss curves.

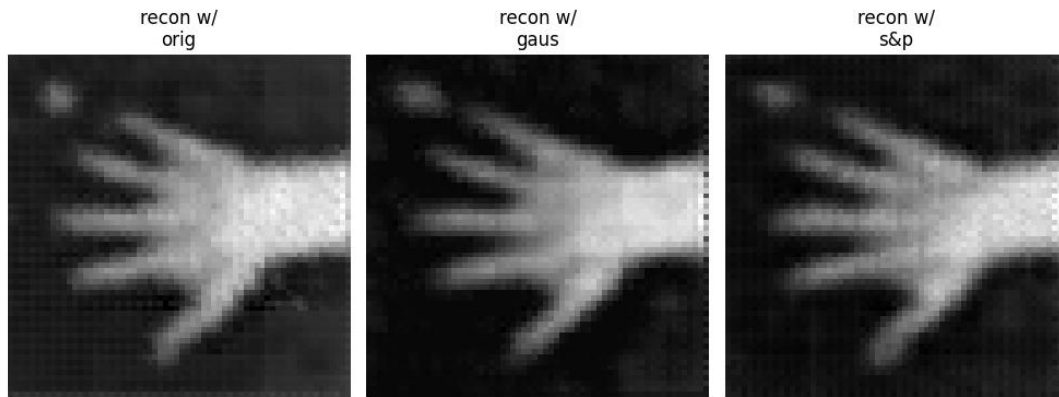
Generate a plot of average loss vs. epoch for each of the data types (orig, gaus, s&p), with different line color representations for each data type and a legend to indicate which line corresponds to which data type.



Step 6. Compare the performance of three AE models.

Retrieve a single image from the testing dataset, pass it through each trained AE model (trained on different noise types), collect the reconstructed images, and then display the original and reconstructed images side by side for comparison.





(3) Training workflow of the GAN

Step 1. Initialize the GAN model architecture.

Discriminator

Input Shape:

The network is designed to process images with a single channel (grayscale) and spatial dimensions of 64x64 pixels. The tuple **(1, 64, 64)** specifies the format as (channels, height, width), making it suitable for handling 2D image data.

Channels:

This specifies the number of output filters for each convolutional layer in the Discriminator. The first layer produces **8** feature maps, the second layer produces **16**, the third layer produces **32**, the fourth layer produces **64**, and the final layer reduces the output to a single feature map. Increasing the number of channels at each layer enables the network to learn more complex and abstract features as the image is processed.

Strides:

It determines the step size for each convolutional layer. A stride of **2** indicates that the kernel moves 2 pixels at a time, effectively halving the spatial dimensions (height and width) after each layer. The final layer employs a stride of 1, which maintains the spatial dimensions of the last feature map. This progressive down-sampling compresses the input image while preserving essential features.

Number of Residual Units:

A residual unit typically incorporates shortcut connections that bypass certain layers, which facilitates the training of deeper networks by alleviating the vanishing gradient problem. Setting it to **1** adds a single residual block, thereby enhancing feature learning while maintaining a relatively simple model.

Kernel Size:

This specifies the size of the convolutional kernel for each layer. A kernel size of **5** indicates that each convolutional filter will have a spatial dimension of 5x5, enabling it to capture patterns and features over a slightly larger area. This can be particularly effective for detecting more complex patterns in the input image.

Generator**Latent Size:**

This specifies the input size to the generator, with the **latent vector** set to **64**. The latent vector, typically composed of **random noise**, serves as the starting point from which the generator creates images. It represents the compressed, high-level features that the generator decodes into a complete image, enabling the model to map from a low-dimensional space to a higher-dimensional image space.

Start shape:

It defines the initial shape of the output from the first layer of the Generator. The output begins with **64** feature maps, each having an **8x8** spatial dimension. This serves as the foundation for the upsampling process, during which the Generator gradually increases the spatial dimensions while decreasing the number of channels to produce the final image.

Channels:

This determines the number of output channels at each layer of the generator. The network begins with **32** channels, then reduces to **16**, followed by **8**, and ultimately outputs a single channel. This progressive decrease in the number of channels aids in

reconstructing the image while simplifying the complexity of features. The final single-channel output corresponds to a grayscale image.

Strides:

It specifies the stride for upsampling (transposed convolution) at each layer. A stride of **2** doubles the spatial dimensions (height and width) of the output at each step, effectively increasing the resolution of the generated image. The final stride of 1 maintains the spatial dimensions, ensuring that the output retains the desired image size without further scaling.

Step 2. Initialize the GAN model weights.

Set the initial weights of the model layers in both the Discriminator and Generator using **values sampled from a normal distribution**, typically with a mean of 0 and a small standard deviation. Proper initialization can help prevent issues such as vanishing or exploding gradients, resulting in faster convergence and more stable training.

Step 3. Add a **Sigmoid activation function** to the generator.

The purpose is to ensure that the output of the generator is constrained to the range [0, 1], which matches the scaling of the input image.

Step 4. Select the appropriate loss function and optimizer.

Binary Cross-Entropy (BCE) Loss:

Measure the binary difference between predicted outputs and target labels. This method is used to classify images as either **real (label = 1)** or fake (**label = 0**).

Discriminator Loss: This metric evaluates the effectiveness of the discriminator in distinguishing between real images and generated ones.

Generator Loss: This metric evaluates how effectively the generator tricks the discriminator into classifying fake images as real.

Formula

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where N represents the total number of samples, while y_i denotes the true label for the i -th sample, which is either 0 or 1. The predicted probability for the i -th sample, $\hat{y}_i$, ranges between 0 and 1. When the true label y_i is 1, the loss is calculated as $-y_i \log(\hat{y}_i)$, encouraging $\hat{y}_i$ to be close to 1. Conversely, when y_i is 0, the loss becomes $-(1 - y_i) \log(1 - \hat{y}_i)$, encouraging $\hat{y}_i$ to be close to 0.

Advantages

Effective for Binary Classification: Ideal for tasks that require binary output, such as GANs, logistic regression, and binary classification.

Gradient-Based Optimization: This method provides smooth and differentiable gradients, making it suitable for backpropagation and optimization algorithms such as Adam or SGD.

Penalty for Confidence Misalignment: The system imposes greater penalties on predictions when a model is confidently incorrect, thereby encouraging the model to concentrate on accurately estimating classification probabilities.

Support for Imbalanced Data: It can be adjusted using weighted BCE to address class imbalances by assigning greater weight to the minority class.

Compatibility with Sigmoid Activation: The Sigmoid activation function is often used to ensure that output values are constrained between 0 and 1.

The optimizer utilizes the same **Adam optimizer** as the AE model; however, the **learning rate** for the GAN is set to **0.0002**, and additional **beta** parameters of **(0.5, 0.999)** are specified.

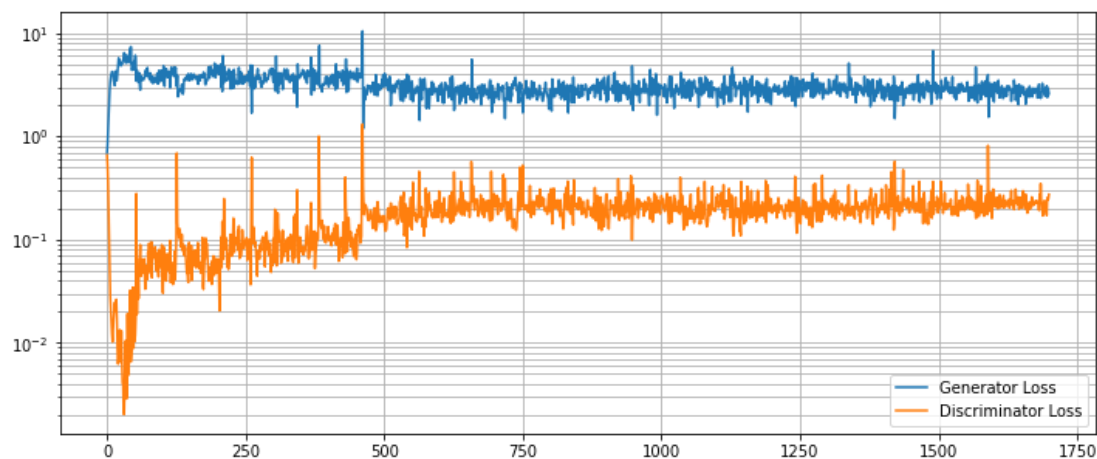
Step 5. Training loop for the GAN model.

The generator and discriminator networks were iteratively updated using the dataset, running for a total of **100 epochs**. In each batch, the generator is trained to improve its ability to deceive the discriminator, while the discriminator is trained five times for each

generator update to better differentiate between real and generated images. This approach ensures a balanced competition between the two networks. The losses of both the generator and discriminator are recorded at each training step, providing valuable data for analyzing training progress. This structure enables the generator to steadily enhance the quality of its output while the discriminator refines its detection accuracy.

Step 6. Plot loss curves.

The individual loss values for the generator and discriminator can be graphed together. These values should reach an equilibrium as the generator's ability to deceive the discriminator balances with the network's capacity to accurately distinguish between real and fake images.



Step 7. Visualize a collection of generated images.

We hope that most generated images will have four fingers and a thumb, as expected, assuming that polydactyl examples are not present in large numbers in the dataset.



IV. Expected Outcomes

For AE Model:

Calculate the MSE loss between the reconstructed images and the original images for the three AE models: original, Gaussian, and salt & pepper, using the testing dataset.

These values were subsequently stabilized to achieve convergence by adjusting the hyperparameters.

For GAN Model:

The Fréchet Inception Distance (FID) is computed between the training images and the generated images to evaluate the performance of the generator. The GAN is then fine-tuned by adjusting hyperparameters to minimize this value.

References

<https://docs.monai.io/en/0.5.2/highlights.html>

<https://github.com/Project-MONAI/tutorials>

<https://github.com/Project-MONAI/MONAILabel>

https://docs.monai.io/en/1.2.0/_modules/monai/networks/nets/autoencoder.html

<https://medium.com/pytorch/monai-v0-6-and-monai-label-v0-1-e738556b0a10>

https://fedbiomed.org/latest/tutorials/monai/01_monai-2d-image-classification/

<https://medium.com/pytorch/monai-starts-to-explore-learning-based-medical-image-registration-ab6b143840b7>

[https://github.com/Project-](https://github.com/Project-MONAI/tutorials/blob/main/modules/autoencoder_mednist.ipynb)

[MONAI/tutorials/blob/main/modules/autoencoder_mednist.ipynb](https://github.com/Project-MONAI/tutorials/blob/main/modules/autoencoder_mednist.ipynb)

[https://github.com/Project-](https://github.com/Project-MONAI/tutorials/blob/main/modules/mednist_GAN_tutorial.ipynb)

[MONAI/tutorials/blob/main/modules/mednist_GAN_tutorial.ipynb](https://github.com/Project-MONAI/tutorials/blob/main/modules/mednist_GAN_tutorial.ipynb)